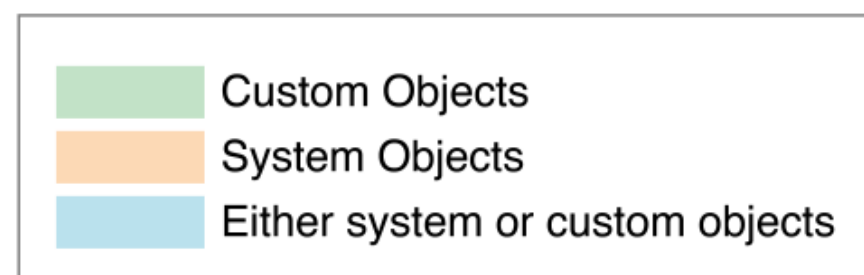
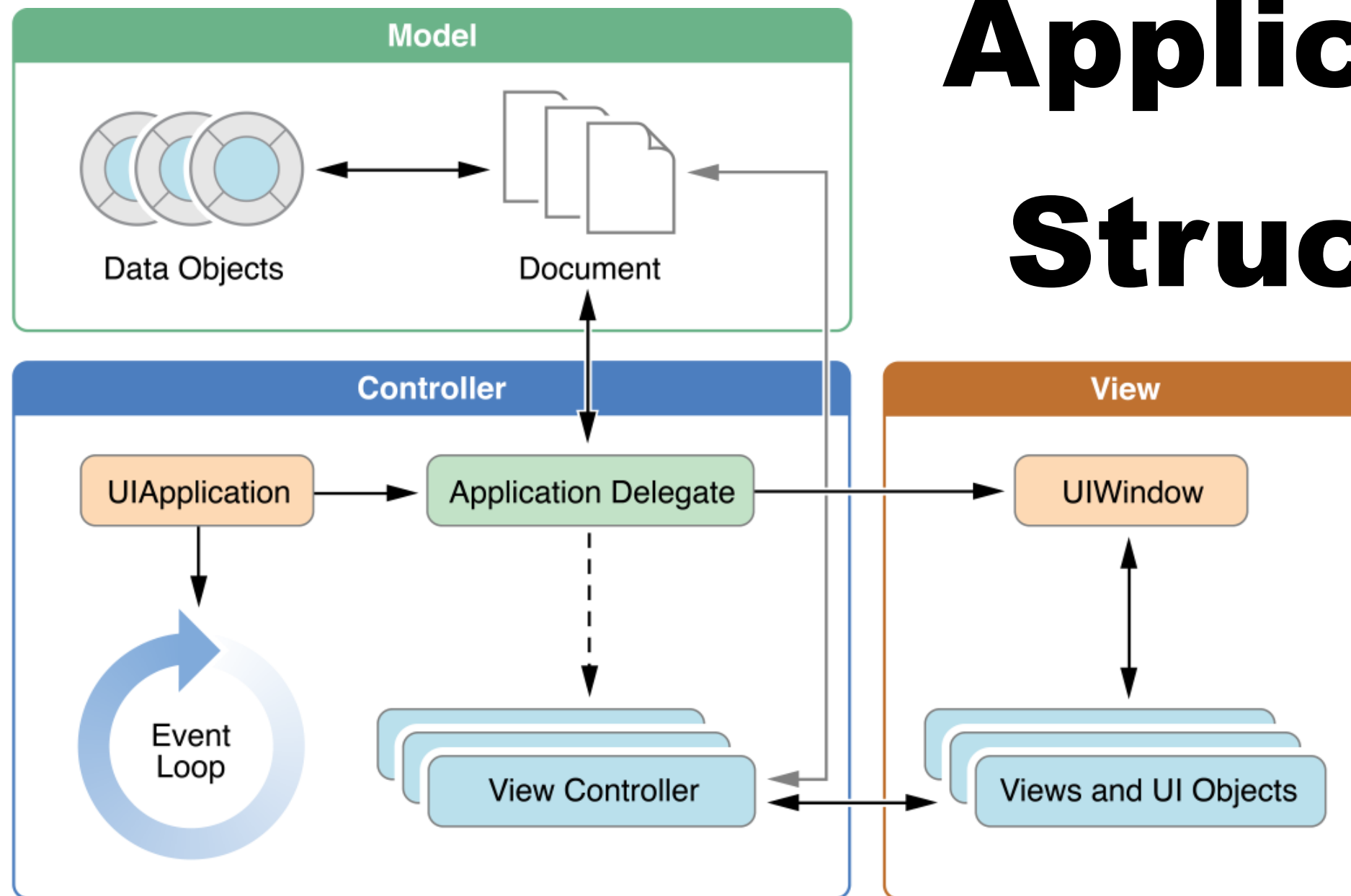


App Lifecycle

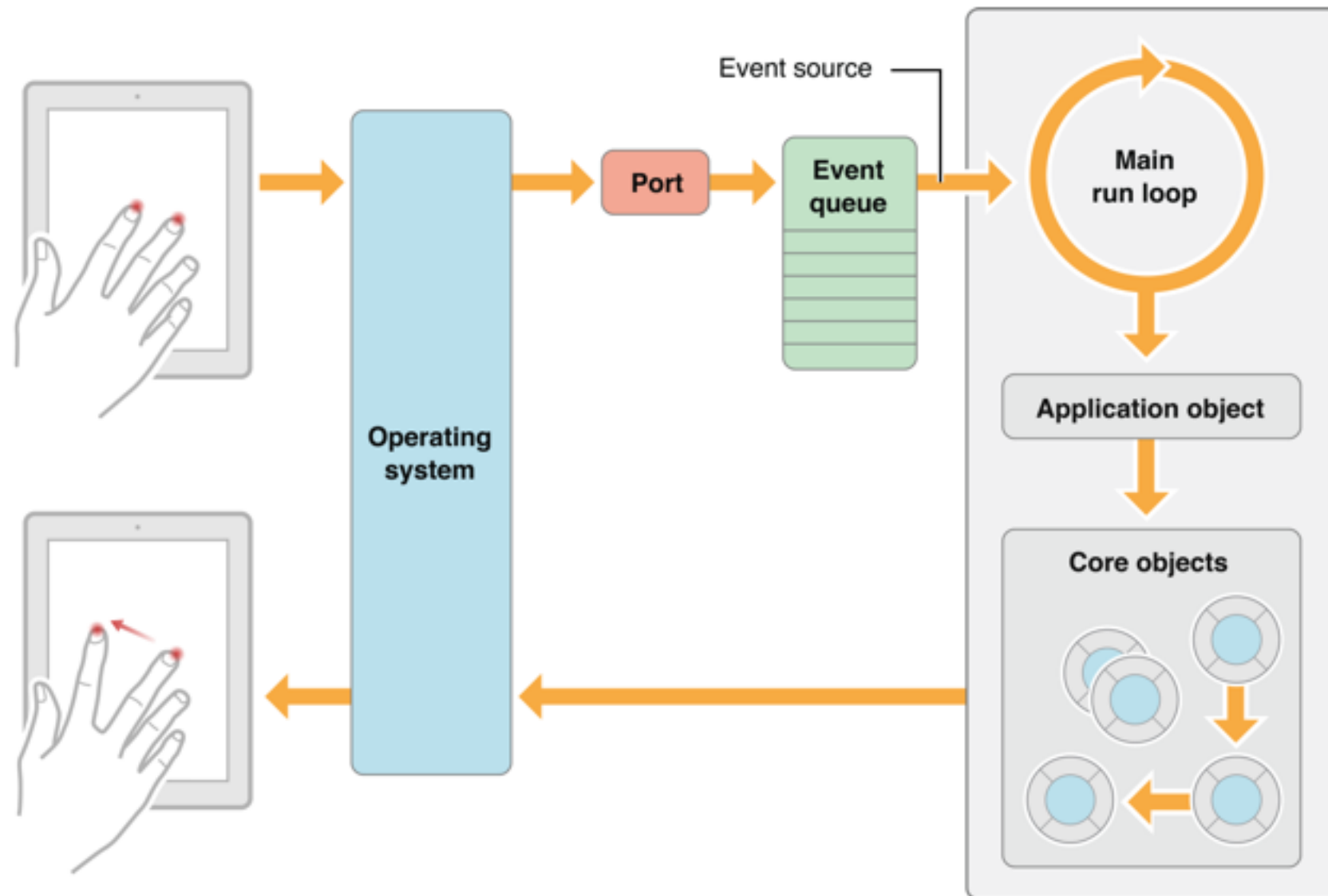
Application Structure



Run Loops

- A loop a thread enters and uses to run event handlers in response to incoming events
- Used to schedule work and coordinate the receipt of incoming events
- Keeps a thread busy when there is work to do and puts it to sleep when there is none

Event Queue & Run Loop



The Main Run Loop

- Each App has one
- Processes all user-related events.
- The [UIApplication](#) object sets it up at launch time
- Executes on the app's main thread
 - Ensures that user-related events are processed serially in the order in which they were received
- Events are queued internally by the app and dispatched one-by-one to the main run loop for execution.
- A touch event is usually dispatched to the main window object, which in turn dispatches it to the view in which the touch occurred

UIApplication

UIApplication: at the heart of every iOS application

- Facilitates the interactions between the system and other objects in the app
- Manages the event loop and other high-level app behaviors
- It also reports key app transitions and some special events (such as incoming push notifications) to its delegate (**AppDelegate**)

AppDelegate

- aka [UIApplicationDelegate](#)
- Receives messages from UIApplication about **app state transitions**
- Often used to set up initial data structures or a “session” model of some sort (will discuss later)
- Holds a reference to the main UIWindow (iOS 12 and below)

SceneDelegate

- aka [UISceneDelegate](#)
- Introduced in iOS 13
- Shares responsibilities of AppDelegate
- Crucial to SwiftUI
- Receives messages from UIApplication about **app state transitions**
- Often used to set up initial data structures or a “session” model of some sort (will discuss later)
- Holds a reference to the main UIWindow (iOS 12 and below)

App Delegate Methods

- **applicationDidFinishLaunchingWithOptions:** basically the “main” function of your app
- **applicationDidBecomeActive:** Your app is about to become the foreground app. Use this method for any last minute preparation.
- **applicationWillResignActive:** Your app is transitioning away from being the foreground app. Use this method to save any needed data, invalidate timers, etc.
- **applicationDidEnterBackground:** entered background and could be suspended at any time. About 5 seconds to return but can request additional small increment of time.
- **applicationWillEnterForeground:** Your app is moving out of the background and back into the foreground, but is not yet active.
- **applicationWillTerminate:** Your app is being terminated. This method is not called if your app is already suspended. About 5 seconds max to return.

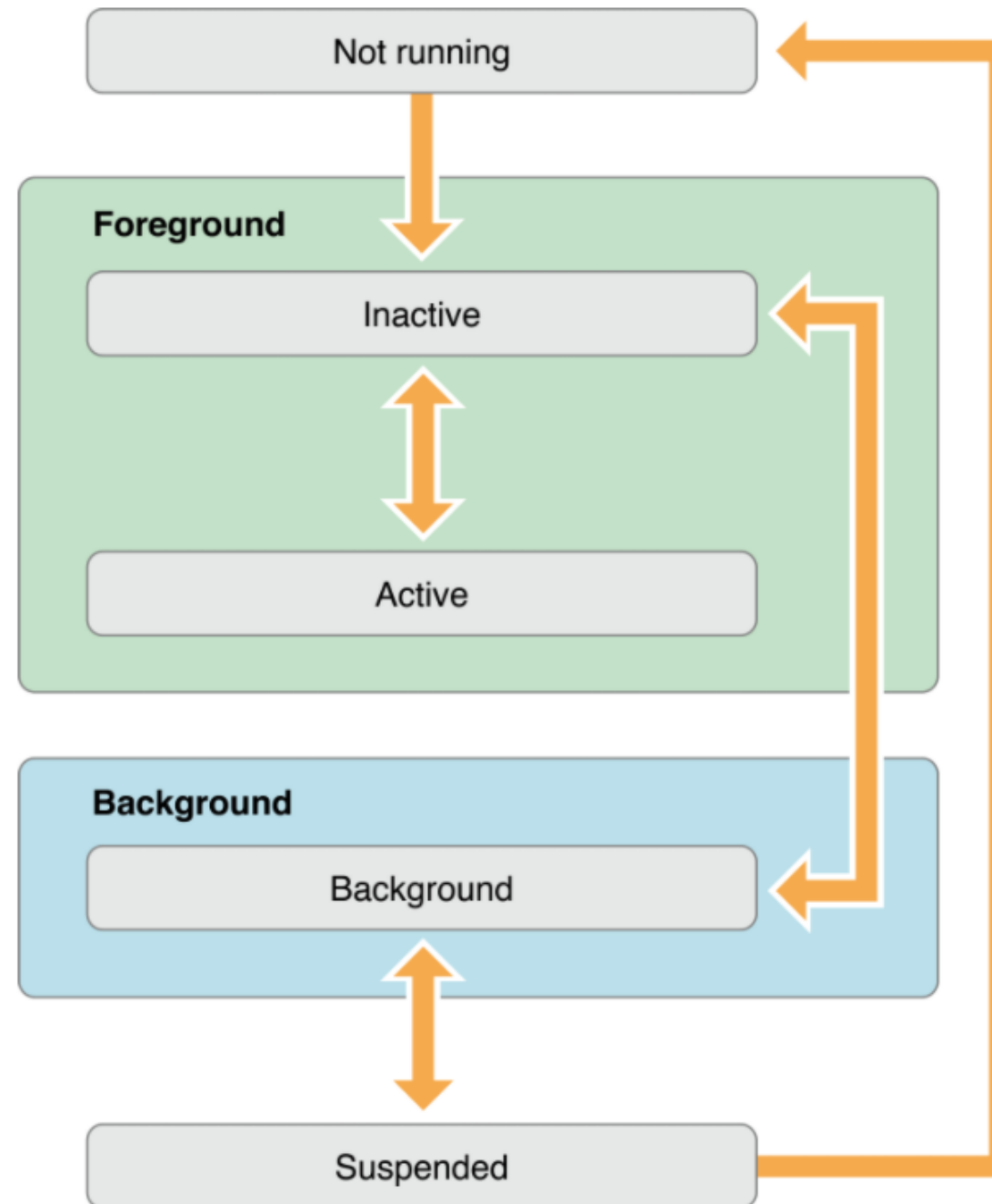
App State

- Determines what it can and cannot do at any time.
- Example:
 - A foreground app has the user's attention, so it has priority over system resources, including the CPU
 - By contrast, a background app must do as little work as possible, and preferably nothing, because it is offscreen
- When state changes, UIKit notifies you by calling methods of the appropriate delegate object:
 - In iOS 13 and later, use [UISceneDelegate](#) objects to respond to life-cycle events in a scene-based app.
 - In iOS 12 and earlier, use the [UIApplicationDelegate](#) object to respond to life-cycle events.

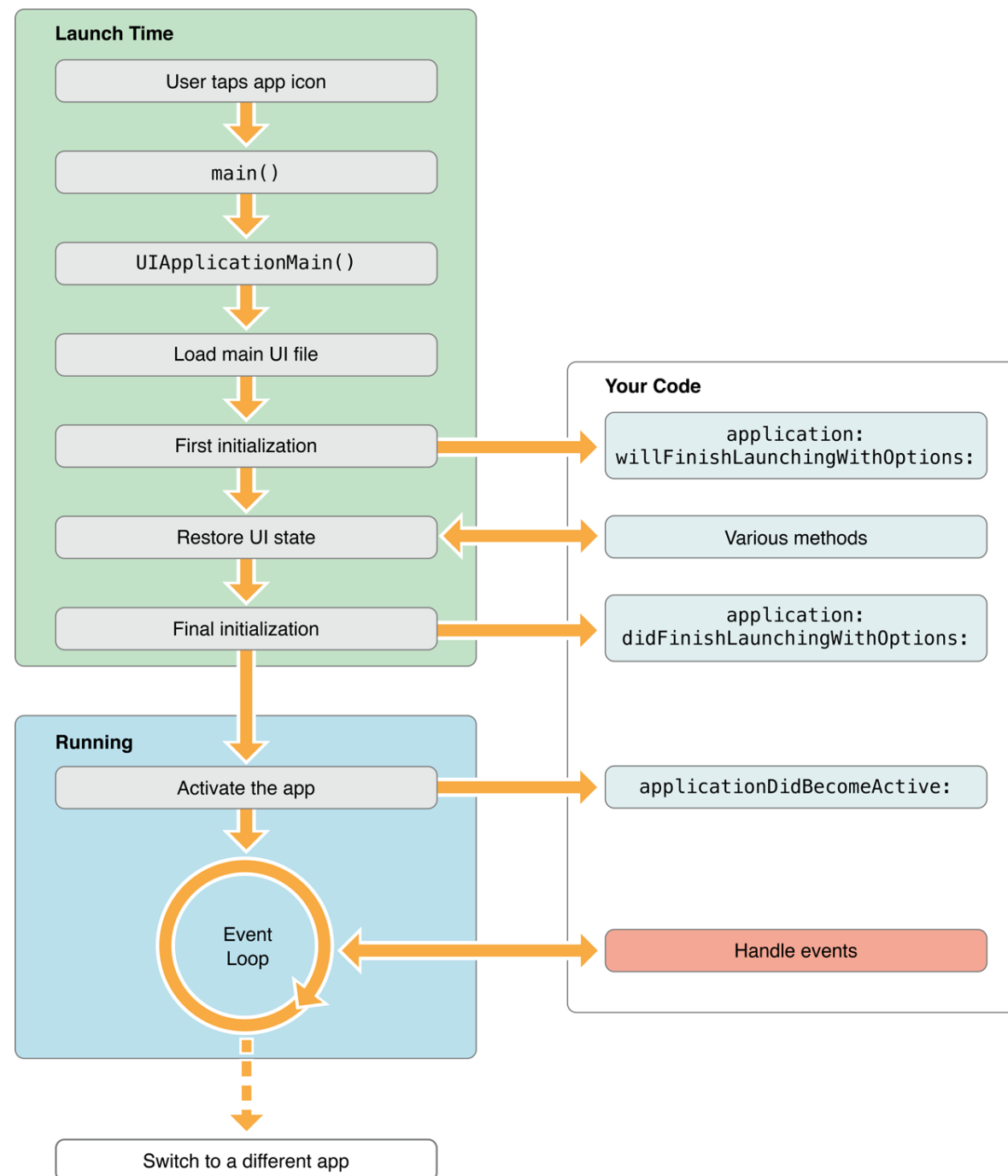
App States

- **Not Running:** not started yet or was terminated
- **Inactive:** In foreground but not yet receiving events (transitional)
- **Active:** normal foreground state and receiving events
- **Background:** in background and still executing code. Usually transitional on way to being suspended. Apps can request ability to run in background.
- **Suspended:** in background and not executing. System does this automatically without notifying the app. App remains in memory. In low-memory conditions, system may purge suspended apps without notice.

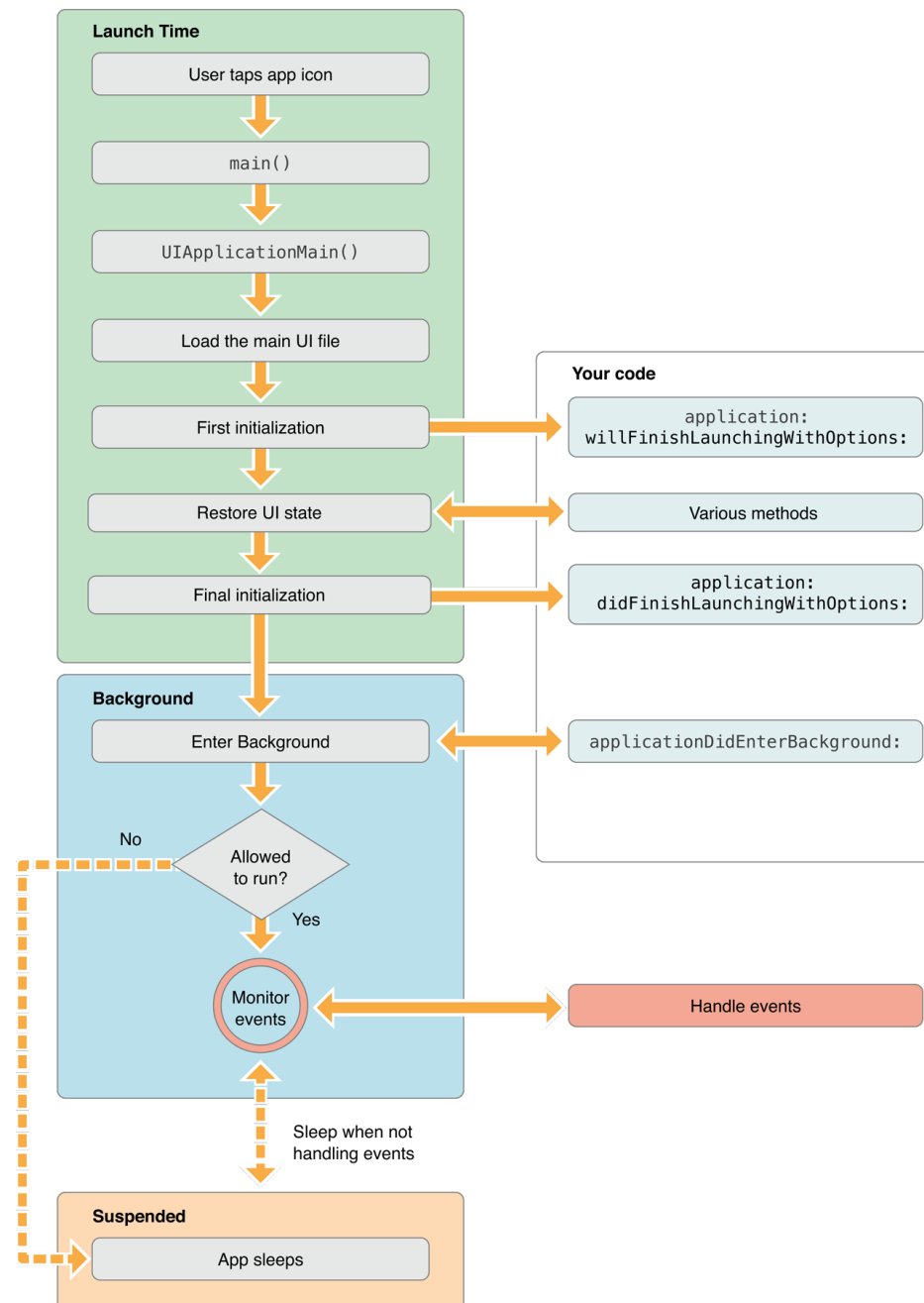
App States



Launching App Into Foreground



Launching App Into Background



Moving From Foreground to Background

